



Denial-of-Service Packet Dropouts in TCP-like Stochastic Linear Control Systems

ELE419 Laboratory Assignment Results Report

Nijat Badalov

Registration Number: 250216213

Module: ELE419 Cybersecurity for Control Systems

School of Electrical and Electronic Engineering

The University of Sheffield

Spring 2026

1. Task, model and control objective

A denial-of-service (DoS) attack on a networked control loop withholds packets rather than altering them, so for some steps the controller receives no measurement or the actuator receives no command. This is an availability problem, and it is a realistic one: in wireless networked control, packet loss is routine and is often masked by model-based prediction in the estimator (Silva and Santos, 2023). The task of this laboratory is to quantify how such losses degrade an optimal loop for a scalar unstable plant, and to find the packet-arrival probabilities for which the loop remains stable. The plant and the loss model follow the lossy-network formulation of Schenato et al. (2007), in which both sensor and actuator packets are delivered only with a given probability:

$$x_{k+1} = Ax_k + \nu_k Bu_k + w_k, \quad y_k = \gamma_k Cx_k + z_k, \quad k = 0, \dots, N-1.$$

The control input reaches the plant only on the steps where its packet arrives ($\nu_k = 1$); the measurement updates the estimate only on the steps where its packet arrives ($\gamma_k = 1$). The arrivals are independent Bernoulli with means $\bar{\nu}$ (actuation) and $\bar{\gamma}$ (sensing); w_k and z_k are zero-mean noises with variances Q and R . A lower arrival probability represents a more aggressive attack.

The controller minimises the finite-horizon LQG cost, adapted so that control effort is charged only on the steps where its packet is delivered:

$$J = \mathbb{E} \left[x_N^2 Q_{xx} + \sum_{k=0}^{N-1} (x_k^2 Q_{xx} + \nu_k u_k^2 Q_{uu}) \right], \quad Q_{xx} = Q_{uu} = 1.$$

The factor ν_k on the input term means that control effort lost to a dropped packet is not charged to the controller. Both plants studied here have $A > 1$, so the open loop is unstable and the controller must actively regulate the state; this is what makes the loss of either channel consequential, since dropped packets remove the feedback that holds an otherwise diverging state near the origin. The two channels are kept separate throughout because they act on different signals: a low $\bar{\nu}$ starves the plant of control, while a low $\bar{\gamma}$ starves the estimator of measurements. Exercise 1 measures the performance loss across six fixed scenarios, and Exercise 2 locates the $(\bar{\nu}, \bar{\gamma})$ region of mean-square stability.

2. TCP-like estimation and MATLAB implementation

The protocol is TCP-like: each transmission is acknowledged, so the controller knows whether its previous command was delivered, and the estimator can condition on the packet-arrival history. This is the setting in which the separation principle holds and the optimal LQG controller is a linear function of the state estimate for lossy networks (Schenato et al., 2007). Without acknowledgement (the UDP-like case) the optimal controller is in general nonlinear and the separation fails (Schenato et al., 2007; Liu, Zhang and Liang, 2024); that case is outside this study and is noted only for contrast.

This assumption has a consequence that is used directly in Exercise 2. Because the actuation outcome is known one step later, it does not appear as a random multiplier in the covariance recursion, so the estimator design depends only on the sensing channel and the controller design only on the actuation channel. The two problems decouple, the optimal controller is the certainty-equivalent linear law, and the stability region therefore factors into one condition per channel rather than a coupled trade-off (Schenato et al., 2007). Under a UDP-like protocol this factorisation is lost and the optimal controller is no longer linear in general (Liu, Zhang and Liang, 2024), which is why the acknowledged TCP-like setting is the one adopted here.

The state estimate is propagated by a Kalman filter whose correction is gated by the sensing arrival and whose prediction uses the known previous actuation outcome ν_{k-1} :

$$\hat{x}_k^- = A\hat{x}_{k-1} + \nu_{k-1}Bu_{k-1}, \quad P_k^- = AP_{k-1}A^\top + Q;$$

$$\gamma_k = 1 : K_k = P_k^- C^\top (CP_k^- C^\top + R)^{-1}, \quad \hat{x}_k = \hat{x}_k^- + K_k(y_k - C\hat{x}_k^-), \quad P_k = P_k^- - K_k CP_k^-; \quad \gamma_k = 0 : \hat{x}_k = \hat{x}_k^-, \quad P_k = P_k^-.$$

When a measurement arrives the estimate is corrected by the Kalman gain; when it does not, the estimate and its covariance are propagated by the model alone. Because the delivered input ν_{k-1} is known, the actuation channel does not enter the covariance recursion, which is exactly why estimation and control separate in the TCP-like case.

Sensor packet loss is therefore an intermittent-observation problem, for which the expected error covariance of an unstable system stays bounded only above a critical sensor arrival probability (Sinopoli et al., 2004). The control law is $u_k = -L\hat{x}_k$, with the gain L and value matrix S taken from the modified Riccati recursion

$$L = (B^\top SB + Q_{uu})^{-1} B^\top SA, \quad S = A^\top SA + Q_{xx} - \bar{\nu} A^\top SBL,$$

in which the actuation arrival probability $\bar{\nu}$ discounts the value of a command by the chance that it is delivered; the existence of a bounded stabilising solution is itself governed by a critical actuation arrival probability (Schenato et al., 2007). Using the steady-

state gain rather than the time-varying finite-horizon gain was checked against the forward Riccati recursion of the provided implementation: for $A = 1.1$ the steady-state gain is $L = 0.703$, and the forward recursion matches it to within 1×10^{-4} after five steps, so the fixed gain is consistent with the supplied code while giving one controller for the repeated trials.

The provided files `DoS_control_TCP_26.m` and `KalmanTCP.m` were used as the starting point and adapted to the assignment cases; the numerical cases follow the ELE419 laboratory specification. The simulation uses base MATLAB only and a fixed random seed (2026), so the figures and tables are reproducible. Exercise 1 uses $A = 1.1$, $B = 1$, $C = 1$, $N = 100$ and $\text{Var}(w) = 2$ with 300 Monte Carlo trials per case; Exercise 2 uses $A = 1.255$, $B = 1$, $C = 1$, $N = 100$, $\text{Var}(w) = 0.5$, $\text{Var}(z) = 0.15$, swept over $(\bar{\nu}, \bar{\gamma})$ with 80 trials per grid point. The complete scripts are in Appendix A and Appendix B.

3. Exercise 1: simulation setup and Monte Carlo summary

The six scenarios switch measurement noise and the two arrival probabilities on and off in turn, so that each communication failure can be read separately. Table 1 lists the cases and their Monte Carlo metrics, and Figure 1 compares the RMS state across the cases on a logarithmic axis. The RMS estimation error $x_k - \hat{x}_k$ is reported alongside the cost, because it distinguishes a sensing failure from an actuation failure far more clearly than the cost on its own.

Three metrics are reported, each computed per realisation and then averaged over the 300 trials. The mean cost is the realised LQG cost divided by the horizon, J/N , and measures overall regulation effort and deviation. The RMS state is the root-mean-square of x_k over the horizon and measures how far the plant is held from the origin. The RMS estimation error is the root-mean-square of $x_k - \hat{x}_k$ and isolates the estimator from the controller, which is what makes the actuation-versus-sensing distinction visible. The realised actuation and sensing rates were also checked against the target $\bar{\nu}$ and $\bar{\gamma}$ to confirm that the Bernoulli model behaved as intended.

Table 1. Exercise 1 Monte Carlo summary (means over 300 trials; $A = 1.1$, $N = 100$, $\text{Var}(w) = 2$).

Case	$\bar{\nu}$	$\bar{\gamma}$	$\text{Var}(z)$	mean cost	RMS state	RMS error	interpretation
1	1	1	0	3.55	1.53	0.00	healthy benchmark; exact estimate
2	1	1	1	4.59	1.81	0.85	healthy; small estimation error from sensor noise
3	0.05	1	0.5	2.1×10^5	112	0.64	actuation starved; estimate still accurate
4	1	0.05	0.5	6.3×10^5	208	188	blind control on a degraded estimate
5	0	1	0.1	9.1×10^7	7.5×10^3	0.31	observer-only; no control authority
6	1	0	0.1	9.9×10^7	7.7×10^3	7.0×10^3	blind control; estimate diverges

Mean cost is the per-step realised cost J/N . For the four attacked cases the cost is heavy-tailed (its standard deviation exceeds its mean), so those values are orders of magnitude rather than exact figures.

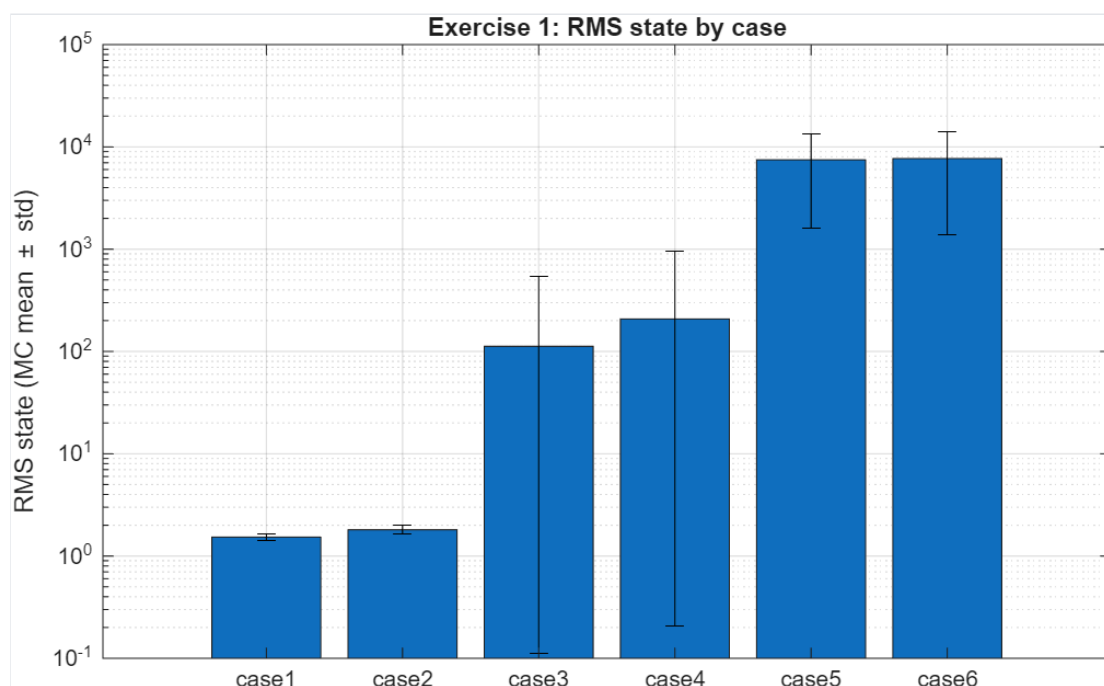


Figure 1. RMS state by scenario (Monte Carlo mean and standard deviation, logarithmic axis). The two healthy cases stay near 1.5–1.8, while the four attacked cases rise by two to four orders of magnitude.

4. Exercise 1: interpretation of the six cases

Cases 1 and 2 are the healthy references. With a noiseless measurement (Case 1) the filter recovers the state exactly, so the RMS estimation error is zero and the mean cost is 3.55. Adding sensor noise of variance 1 (Case 2) introduces a steady estimation error of 0.85, close to the steady-state Kalman value, and raises the cost only to 4.59 — a 29% increase with no packets lost. Both cases keep the two arrival probabilities at one, well above the critical value of $1 - 1/A^2 \approx 0.174$ for $A = 1.1$, so the loop stays mean-square stable.

Cases 3 and 4 each deny one channel 95% of the time and separate the two failures (Figure 2). Under actuation loss (Case 3) sensing is intact, so the estimate stays accurate (RMS error 0.64), but the commands rarely reach the plant; the state runs largely open-loop and its RMS reaches 112. Under sensing loss (Case 4) the actuator works, but the estimator runs mostly on prediction, its RMS error grows to 188, and the actuator then applies those wrong commands — so the RMS state (208) and the cost (6.3×10^5) are both higher than in Case 3. The comparison is the central observation of Exercise 1: actuation loss removes control authority while leaving perception intact, whereas sensing loss damages perception and, through it, control.

Severe single-channel loss: good estimate but no control (Case 3) vs degraded estimate (Case 4)

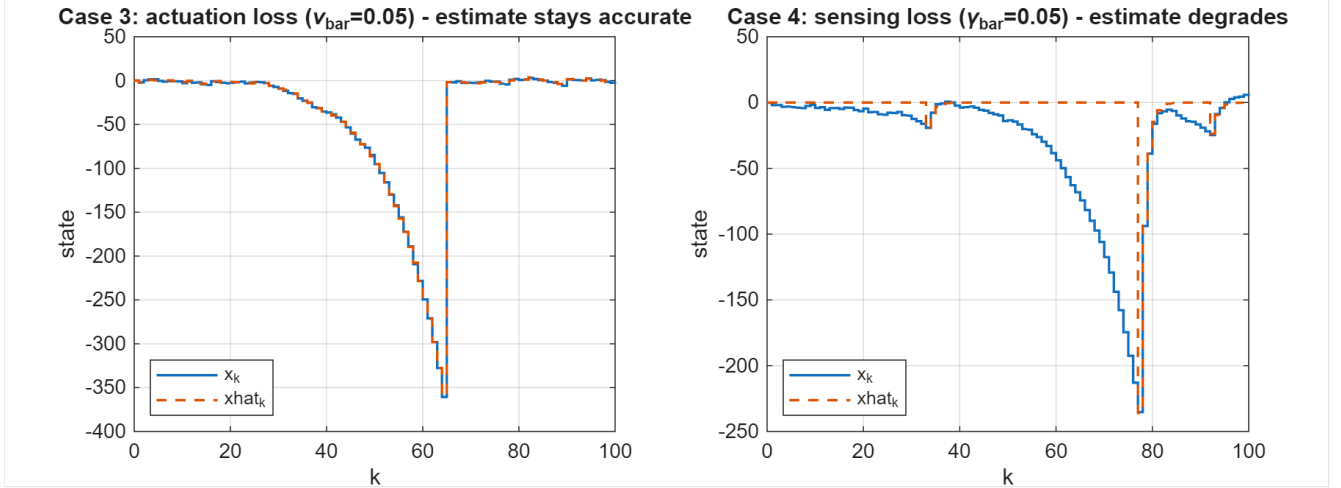


Figure 2. Severe single-channel loss. Left, Case 3 (actuation loss): the estimate stays on the state because sensing is intact, but the state drifts because commands rarely arrive. Right, Case 4 (sensing loss): the estimate separates from the state, and the working actuator then acts on it.

Cases 5 and 6 remove one channel completely and are useful because they fail for opposite reasons (Figure 3). Case 5 is observer-only: with no actuation the plant is open-loop and the state diverges to an RMS of 7.5×10^3 , yet because sensing is perfect the estimate tracks that runaway almost exactly and the RMS estimation error is only 0.31. The state can be estimated but not controlled. Case 6 is blind-control: actuation is available, but the filter never receives a correction, so for an unstable plant the error covariance grows without bound (Sinopoli et al., 2004); the estimate stays near zero while the true state runs away, and the RMS estimation error reaches 7.0×10^3 . The contrast in RMS estimation error between the two cases, about 0.31 against 7.0×10^3 , identifies which channel was attacked even though the RMS state is similar in both. Each of the four attacked cases drives one arrival probability below the 0.174 threshold for $A = 1.1$, which is consistent with the loss of mean-square stability seen in the metrics.

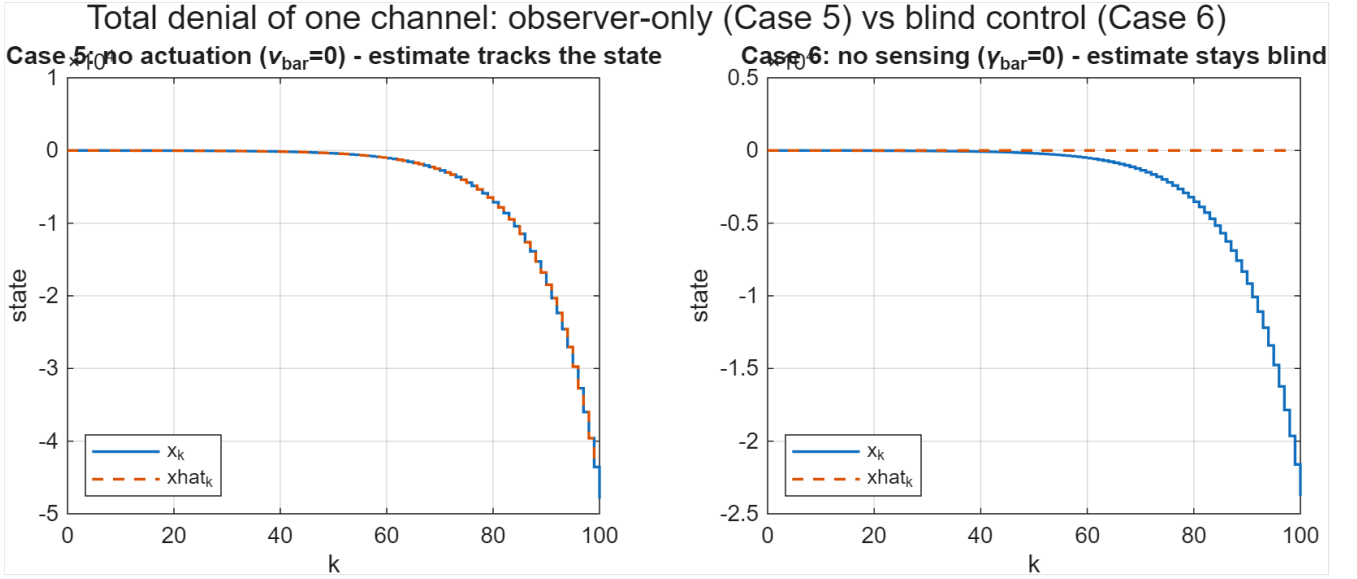


Figure 3. Total denial of one channel. Left, Case 5 (no actuation): the state diverges open-loop but the estimate stays on top of it. Right, Case 6 (no sensing): the state diverges while the estimate, receiving no correction, stays near zero.

Read across Table 1, the numbers quantify the asymmetry between the two channels. Sensor noise alone (Case 1 to Case 2) costs about 29% in mean cost. Moving the same 95% loss from the actuation channel to the sensing channel (Case 3 to Case 4) roughly triples the mean cost, from 2.1×10^5 to 6.3×10^5 , and raises the RMS estimation error from 0.64 to 188 as the estimator loses its corrections. Removing a channel entirely (Cases 5 and 6) adds a further two orders of magnitude, to about 10^8 ; yet the two total-denial cases differ by only about 9% in mean cost while differing by four orders of magnitude in RMS estimation error. Cost on its own therefore cannot tell the two attacks apart, whereas the estimation error separates them immediately, which is the practical value of reporting it alongside the cost.

5. Exercise 2: stability threshold calculation

Exercise 2 uses the more unstable plant $A = 1.255$ with $\text{Var}(w) = 0.5$ and $\text{Var}(z) = 0.15$, and asks for the arrival probabilities that keep the loop mean-square stable. The boundary is taken from the deterministic recursions, which carry no Monte Carlo noise. The control side is the modified Riccati recursion, which has a bounded stabilising solution only when the actuation arrival probability exceeds a critical value (Schenato et al., 2007); the sensing side is the expected error-covariance recursion, which stays bounded only when the sensor arrival probability exceeds the intermittent-observation threshold (Sinopoli et al., 2004). For an unstable scalar plant both critical values take the same closed form, the mean-square stabilisation threshold for networked control under packet dropout (Liang, Xu and Zhang, 2017). Iterating the two recursions to their fixed points,

$$\begin{aligned} S &\leftarrow A^\top SA + Q_{xx} - \bar{\nu} A^\top SB (B^\top SB + Q_{uu})^{-1} B^\top SA, \\ \bar{P} &\leftarrow A \bar{P} A^\top + Q - \bar{\gamma} A \bar{P} C^\top (C \bar{P} C^\top + R)^{-1} C \bar{P} A^\top, \end{aligned}$$

locates the boundary directly: the control value S stays finite only for $\bar{\nu} > 1 - 1/A^2$, and the expected covariance $\bar{P}$ only for $\bar{\gamma} > 1 - 1/A^2$. Figure 4 plots both quantities as their arrival probability is reduced, and each grows sharply and then diverges as the probability approaches 0.365 from above.

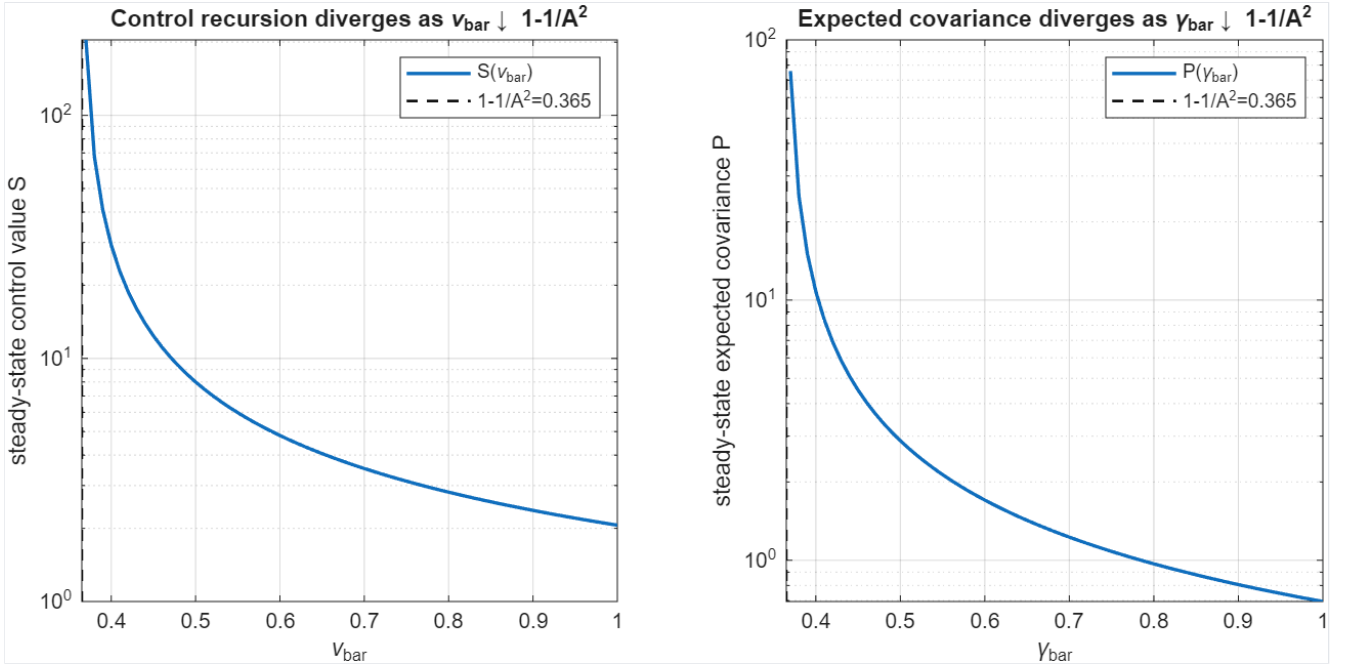


Figure 4. Deterministic recursions used to fix the threshold. Left: steady-state control value versus $\bar{v}$. Right: steady-state expected covariance versus $\bar{\gamma}$. Each diverges as its arrival probability approaches $1 - 1/A^2 = 0.365$ (dashed) and has no finite value below it.

For $A = 1.255$: $p_{\text{crit}} = 1 - 1/A^2 = 1 - 1/(1.255^2) \approx 0.365$. The deterministic result is therefore that mean-square stability requires $\bar{v} > 0.365$ and $\bar{\gamma} > 0.365$; if either arrival probability falls below about 0.365 the loop is unstable.

Table 2 lists the thresholds. The deterministic actuation and sensing thresholds both come out at 0.365, matching the analytic value. Because the TCP acknowledgement keeps $\bar{v}$ out of the covariance recursion, the two conditions are independent and the stable set is the rectangle $\bar{v} > 0.365$, $\bar{\gamma} > 0.365$, rather than a coupled trade-off. The Monte Carlo sweeps support the same boundary but place it lower (0.19 to 0.40, depending on channel and horizon); this is expected, because the second moment is heavy-tailed and a finite sample over a finite horizon underestimates it, with rare divergent trajectories dominating the mean. The Monte Carlo result is therefore used as supporting evidence, not as the exact stability boundary.

Table 2. Exercise 2 stability thresholds ($A = 1.255$, with $1 - 1/A^2 = 0.365$).

Quantity	Value
Analytic critical arrival probability $1 - 1/A^2$	0.365
Deterministic actuation threshold (control Riccati recursion)	0.365
Deterministic sensing threshold (expected covariance recursion)	0.365
Monte Carlo actuation threshold ($N = 100 / 300$)	0.19 / 0.29
Monte Carlo sensing threshold ($N = 100 / 300$)	0.40 / 0.28
Final mean-square stable region	$\bar{v} > 0.365$ and $\bar{\gamma} > 0.365$

6. Exercise 2: stability map and supporting 3D visualisation

Figure 5 is the main quantitative result. The left panel is the deterministic mean-square stable region, a rectangle with its corner at $(0.365, 0.365)$; the right panel is the Monte Carlo mean state energy on the same grid, which reproduces the same stable corner with a softer edge because of the finite-sample effect noted above. The heatmap gives the clearest view of the stable rectangle and of how quickly the energy rises once a channel falls below threshold.

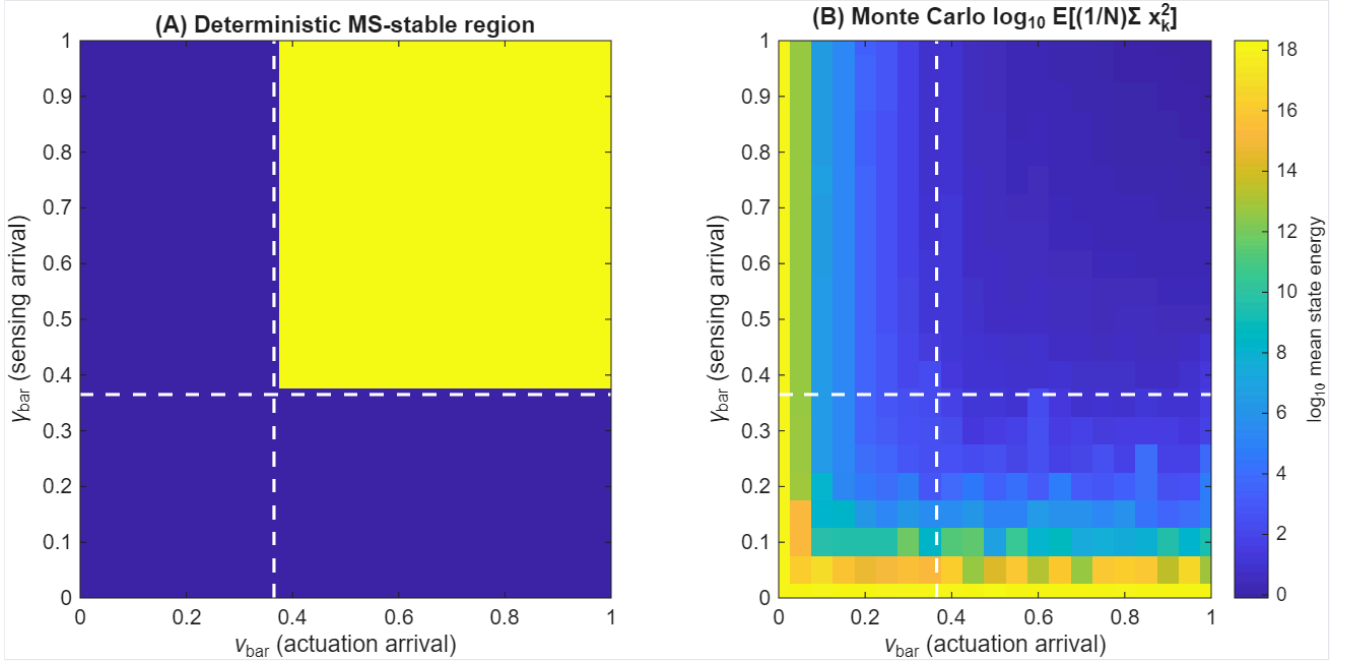


Figure 5. Left: deterministic mean-square stable region (corner at $0.365, 0.365 = 1 - 1/A^2$, dashed lines). Right: Monte Carlo $\log_{10}$ mean state energy on the same grid. The deterministic rectangle is the quantitative result; the Monte Carlo map supports the trend.

Figure 6 shows the same data as a three-dimensional surface. It is included as a visual summary of the transition shown in the heatmap and should not be read as the source of the threshold value: the numerical boundary is taken from the deterministic Riccati and covariance recursions, while the surface only illustrates how the simulated state energy climbs as either arrival probability approaches the loss-dominated region. The surface forms a shallow valley in the corner where both probabilities are high — the stable region — and rises by many orders of magnitude towards either axis.

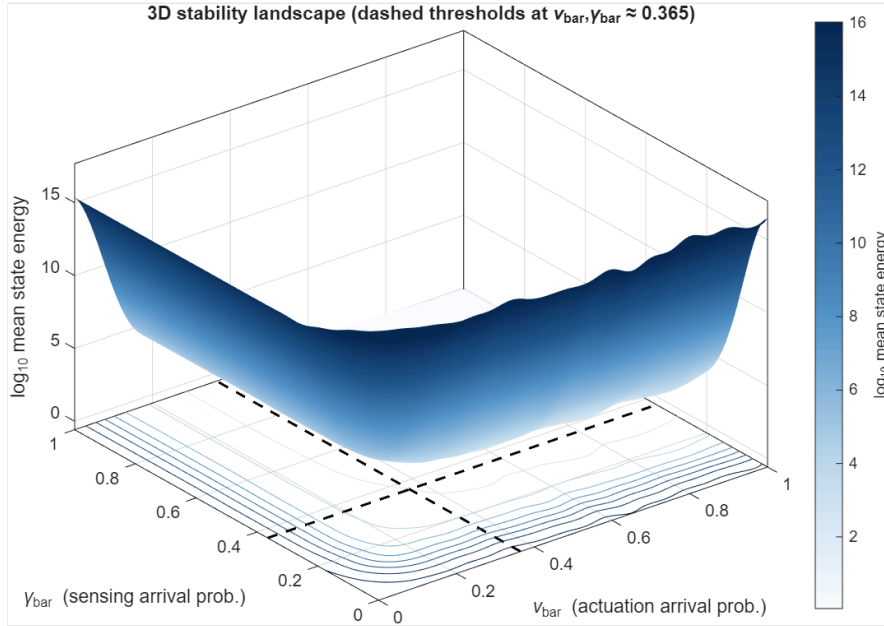


Figure 6. Three-dimensional stability landscape: $\log_{10}$ mean state energy over the arrival-probability grid (same data as Figure 5, lightly smoothed for display). The dashed lines on the base mark the thresholds at about 0.365. The figure is supporting visual evidence; the quantitative threshold comes from the recursions.

The gap between the deterministic threshold of 0.365 and the Monte Carlo estimates of 0.19 to 0.40 is itself worth interpreting. Mean-square stability is a statement about the second moment $\mathbb{E}[x_k^2]$, which for a system with multiplicative packet-loss noise is dominated by rare trajectories that happen to suffer long runs of lost packets. Over a finite horizon and a finite number of trials those rare events are under-sampled, so the empirical state energy sits below the true second moment and the apparent boundary moves inward. Lengthening the horizon from $N = 100$ to $N = 300$ moves the Monte Carlo actuation threshold from 0.19 to 0.29, that is, towards the analytic value, which is consistent with this explanation. The deterministic recursions test boundedness exactly and are therefore used for the reported threshold, with the Monte Carlo sweeps confirming the trend rather than setting the boundary (Schenato et al., 2007).

7. Final engineering conclusions

The six Exercise 1 cases separate the two communication failures. Actuation loss removes control authority while leaving the estimate accurate (Cases 3 and 5, with RMS estimation error 0.64 and 0.31), whereas sensing loss damages the estimate and, through it, the control (Cases 4 and 6, with RMS estimation error 188 and 7.0×10^3). Complete denial of either channel destabilises the unstable plant, and Cases 5 and 6 show the two failure modes most clearly: an observer that cannot act versus an actuator working from a blind estimate. For Exercise 2 the mean-square stable region is $\bar{\nu} > 0.365$ and $\bar{\gamma} > 0.365$, obtained from the deterministic Riccati and covariance recursions and equal to the analytic $1 - 1/A^2$ predicted by the lossy-network theory (Schenato et al., 2007); the Monte Carlo sweeps support this boundary but underestimate it near the threshold. In practical terms both channels must be protected to a comparable arrival rate, here above about 0.365, and the estimation error is a useful indicator of which channel an attack is targeting, since a sensing-channel attack and an actuation-channel attack leave very different traces on the estimate. For a defender this suggests two concrete measures: provision the actuation and sensing links to a similar reliability, because the weaker channel sets the stability margin, and monitor the innovation or estimation-error magnitude as a low-cost online signal that separates a sensing attack from an actuation attack. The analysis assumes an idealised acknowledgement and independent Bernoulli losses; a bursty or correlated attack, or a lossy acknowledgement channel, would shrink the usable region and is the natural next step for this work.

References

- Liang, X., Xu, J. and Zhang, H. (2017) 'Optimal control and stabilization for networked control systems with packet dropout and input delay', *IEEE Transactions on Circuits and Systems II: Express Briefs*, 64(9), pp. 1087–1091. doi: 10.1109/TCSII.2016.2642986.
- Liu, X., Zhang, A. and Liang, X. (2024) 'LQG control for networked control systems with packet dropouts and input delay: UDP case', *Optimal Control Applications and Methods*. doi: 10.1002/oca.3211.
- Schenato, L., Sinopoli, B., Franceschetti, M., Poolla, K. and Sastry, S.S. (2007) 'Foundations of control and estimation over lossy networks', *Proceedings of the IEEE*, 95(1), pp. 163–187. doi: 10.1109/JPROC.2006.887306.
- Silva, C.A.G. da and Santos, E.L. dos (2023) 'A compensation model for packet loss using Kalman filter in wireless network control systems', *Energies*, 16, 3329. doi: 10.3390/en16083329.
- Sinopoli, B., Schenato, L., Franceschetti, M., Poolla, K., Jordan, M.I. and Sastry, S.S. (2004) 'Kalman filtering with intermittent observations', *IEEE Transactions on Automatic Control*, 49(9), pp. 1453–1464. doi: 10.1109/TAC.2004.834121.

Appendix A: MATLAB code for Exercise 1

The following MATLAB script reproduces the Exercise 1 simulations. It is self-contained and runs from a clean session, with its supporting routines defined as local functions.

```
%% Appendix A: MATLAB code for Exercise 1
% ELE419 Cybersecurity for Control Systems
% Nijat Badalov (250216213), University of Sheffield, Spring 2026
%
% Denial-of-service packet dropouts on a scalar TCP-like LQG control loop.
% This script reproduces every Exercise 1 result. The six attack scenarios are
% simulated for  $A = 1.1$ ,  $N = 100$  and process-noise variance 2, using a
% steady-state TCP-like LQG controller together with a TCP-like Kalman filter.
% For each scenario one representative trajectory is plotted, and 300 Monte
% Carlo trials are used for the summary statistics. The parameter values follow
% the ELE419 DoS laboratory brief. Only base MATLAB is used (no toolboxes).
%
% Run from a clean session: >> Appendix_Code_Exercise1

clear; close all; clc;

masterSeed = 2026;    % fixed seed so the figures and table are reproducible
M           = 300;    % Monte Carlo trials per scenario

figDir = fullfile(pwd, 'figures');
if ~exist(figDir, 'dir'); mkdir(figDir); end

% ---- plant and cost (Exercise 1 parameters from the laboratory brief) ----
P.A = 1.1;  P.B = 1;  P.C = 1;  P.N = 100; % scalar dynamics and horizon
P.Qxx = 1;   P.Quu = 1; % LQ state / input penalties
P.Q = 2; % process-noise variance Var(w)
P.P0 = 0.01; % initial error covariance
P.x0 = []; % random  $x_0 \sim N(0, P_0)$  each trial
P.validate = true;

% ---- six scenarios: {name, nu_bar, gamma_bar, Var(z)=R, description} ----
Cs = {
    'case1', 1.00, 1.00, 0.0, 'Ideal: no loss, no measurement noise';
    'case2', 1.00, 1.00, 1.0, 'No loss, measurement noise  $N(0,1)$ ';
    'case3', 0.05, 1.00, 0.5, 'Severe actuation loss, perfect sensing';
    'case4', 1.00, 0.05, 0.5, 'Perfect actuation, severe sensing loss';
    'case5', 0.00, 1.00, 0.1, 'No actuation, sensing available';
    'case6', 1.00, 0.00, 0.1, 'Actuation available, no sensing';
}
nC = size(Cs, 1);

rows = struct([]);
repOuts = cell(1, nC);

for c = 1:nC
    p = P;
    p.nu_bar = Cs{c,2};
    p.gamma_bar = Cs{c,3};
    p.R = Cs{c,4};
    [p.L, ~, p.gain_converged] = tcp_control_gain(p); % steady-state gain once

    % representative trajectory for the per-case figure
    rng(masterSeed + 100*c + 1);
    repOuts{c} = simulate_tcp_case(p);

    % Monte Carlo statistics
    Jr=zeros(1,M); mc=zeros(1,M); rx=zeros(1,M); re=zeros(1,M);
    mx=zeros(1,M); xN2=zeros(1,M); eu=zeros(1,M); eg=zeros(1,M);
    for t = 1:M
        rng(masterSeed + 100*c + t);
        o = simulate_tcp_case(p);
        Jr(t)=o.J_realised; mc(t)=o.mean_cost; rx(t)=o.rms_x; re(t)=o.rms_e;
        mx(t)=o.max_absx; xN2(t)=o.xN2; eu(t)=o.emp_nu; eg(t)=o.emp_gamma;
    end
    flag = stability_flag(median(xN2), mean(rx));

    info.name = Cs{c,1};
    info.title = sprintf('%s: %s  (\\nu=%.2f, \\gamma=%.2f, Var z=%.2g)', ...
        Cs{c,1}, Cs{c,5}, Cs{c,2}, Cs{c,3}, Cs{c,4});
    plot_case_results(repOuts{c}, info, figDir);

    r.Case=Cs{c,1}; r.nu_bar=Cs{c,2}; r.gamma_bar=Cs{c,3}; r.Var_z=Cs{c,4};
    r.mean_nu=mean(eu); r.mean_gamma=mean(eg);
    r.J_final_mean=mean(Jr); r.mean_cost=mean(mc);
    r.rms_x_mean=mean(rx); r.rms_x_std=std(rx);
    r.rms_e_mean=mean(re); r.rms_e_std=std(re);
end
```

```

r.max_absx_mean=mean(mx); r.stability=flag; r.note=Cs{c,5};
if c==1; rows=r; else; rows(c)=r; end

fprintf(['%s | nu=%.2f gamma=%.2f Var(z)=%.2g | RMSx=%.3g RMSe=%.3g ', ...
        'Jmean=%.3g | %s\n'], Cs{c,1},Cs{c,2},Cs{c,3},Cs{c,4}, ...
        mean(rx),mean(re),mean(Jr),flag);
end

% ---- summary table -----
T = struct2table(rows);
writetable(T, fullfile(figDir, 'exercise1_summary.csv'));
disp(T);

% ---- cross-case comparison figures -----
plot_exercise1_comparison(repOuts, rows, Cs, figDir);

% ---- steady-state vs forward Riccati gain (controller validation) -----
validate_steady_state_gain(P); % A = 1.1, nu_bar = 1 representative

fprintf('\nExercise 1 complete. Figures and exercise1_summary.csv saved in: %s\n', figDir);

%% ===== local functions =====

function out = simulate_tcp_case(p)
% Simulate one realisation of the scalar TCP-like LQG loop over horizon p.N:
% x_{k+1} = A x_k + nu_k B u_k + w_k , w_k ~ N(0,Q)
% y_k = gamma_k C x_k + z_k , z_k ~ N(0,R)
% u_k = -L xhat_k (xhat from the TCP-like Kalman filter)
% Time k = 0..N maps to MATLAB index k+1; inputs/measurements use k = 0..N-1.
% The caller seeds rng before each call, so trials are reproducible.

if ~isfield(p,'validate') || p.validate
    assert(p.nu_bar>=0 && p.nu_bar<=1, 'nu_bar must be in [0,1]');
    assert(p.gamma_bar>=0 && p.gamma_bar<=1, 'gamma_bar must be in [0,1]');
    assert(p.N>=2 && p.N==round(p.N), 'N must be an integer >= 2');
    assert(p.Q>=0 && p.R>=0, 'noise variances must be >= 0');
end
A=p.A; B=p.B; C=p.C; N=p.N; Qxx=p.Qxx; Quu=p.Quu;

if isfield(p,'L') && ~isempty(p.L)
    L = p.L;
    if isfield(p,'gain_converged'); gc=p.gain_converged; else; gc=NaN; end
else
    [L,~,gc] = tcp_control_gain(p);
end

% all randomness drawn up front (indices 1..N hold the k = 0..N-1 values)
nu = double(rand(1,N) < p.nu_bar); % actuation arrivals
gamma = double(rand(1,N) < p.gamma_bar); % sensing arrivals
w = sqrt(p.Q)*randn(1,N); % process noise
z = sqrt(p.R)*randn(1,N); % measurement noise
if isempty(p.x0); x0 = sqrt(p.P0)*randn; else; x0 = p.x0; end

x=zeros(1,N+1); x(1)=x0; % states x_0..x_N
xhat=zeros(1,N+1); % posterior estimates xhat_0..xhat_{N-1}
u=zeros(1,N); y=zeros(1,N);
Ppost=zeros(1,N+1); Ppost(1)=p.P0;
Ppred=zeros(1,N+1); Ppred(1)=p.P0;
Kk=zeros(1,N); mu0=0;

% k = 0: prior, gated correction with y_0, then u_0
y(1) = gamma(1)*C*x(1) + z(1);
s0 = C*p.P0*C' + p.R;
if gamma(1)==1 && s0>0
    K0=(p.P0*C')/s0; xhat(1)=mu0+K0*(y(1)-C*mu0); Ppost(1)=p.P0-K0*C*p.P0; Kk(1)=K0;
else
    xhat(1)=mu0; Ppost(1)=p.P0;
end
u(1)=-L*xhat(1);
x(2)=A*x(1)+nu(1)*B*u(1)+w(1);

% k = 1..N-1: TCP predict/correct, control, propagate
for k=1:N-1
    j=k+1;
    y(j)=gamma(j)*C*x(j)+z(j);
    [xhat(j),Ppost(j),~,Ppred(j),Kk(j)] = kalman_tcp_update( ...
        p, y(j), xhat(k), u(k), nu(k), gamma(j), Ppost(k));
    u(j)=-L*xhat(j);
    x(j+1)=A*x(j)+nu(j)*B*u(j)+w(j);
end

ee = x(1:N)-xhat(1:N); % estimation error

```

```

stage = Qxx*x(1:N).^2 + nu.*(Quu*u.^2); % per-step cost
J_run = cumsum(stage) + Qxx*x(2:N+1).^2;
J_run = [Qxx*x(1)^2, J_run];
J_realised = Qxx*x(N+1)^2 + sum(stage);

out.p=p; out.k_state=0:N; out.k_io=0:N-1;
out.x=x; out.xhat=xhat(1:N); out.ee=ee; out.u=u; out.y=y;
out.nu=nu; out.gamma=gamma; out.Ppost=Ppost(1:N); out.K=Kk;
out.J_run=J_run; out.L=L; out.gain_converged=gc;
out.J_realised=J_realised; out.mean_cost=J_realised/N;
out.emp_nu=mean(nu); out.emp_gamma=mean(gamma);
out.rms_x=sqrt(mean(x.^2)); out.rms_e=sqrt(mean(ee.^2));
out.max_absx=max(abs(x)); out.xN2=x(N+1)^2; out.P_final=Ppost(N);
end

function [xhat_upd,P_upd,xhat_pred,P_pred,K] = ...
    kalman_tcp_update(p,y,xhat_prev,u_prev,nu_prev,gamma_cur,P_prev)
% One step of the TCP-like Kalman filter. The previous actuation outcome
% nu_prev is known (TCP acknowledgement), so it enters the prediction
% deterministically; the current sensing arrival gamma_cur gates the
% correction. Division is used instead of inv(); R = 0 is handled safely
% because the innovation covariance C P^- C' + R >= Q > 0 stays positive.
A=p.A; B=p.B; C=p.C; Q=p.Q; R=p.R;
xhat_pred = A*xhat_prev + nu_prev*B*u_prev; % prediction
P_pred = A*P_prev*A' + Q;
innov = C*P_pred*C' + R;
if gamma_cur==1 && all(innov(:)>0) % correction
    K = (P_pred*C')/innov;
    xhat_upd = xhat_pred + K*(y - C*xhat_pred);
    P_upd = P_pred - K*C*P_pred;
    P_upd = 0.5*(P_upd+P_upd. ');
else % no measurement
    K = zeros(size(P_pred*C'));
    xhat_upd = xhat_pred;
    P_upd = P_pred;
end
end

function [L,S,converged] = tcp_control_gain(p)
% Steady-state TCP-like LQG control gain from the modified Riccati recursion
% L = (B'SB+Quu)^{-1} B'SA , S = A'SA + Qxx - nu_bar A'SB L .
% Iterated to its fixed point. For nu_bar below 1 - 1/A^2 the recursion has no
% finite fixed point (S grows without bound) but the gain still converges to
% A/B; converged is then returned false.
A=p.A; B=p.B; Qxx=p.Qxx; Quu=p.Quu; nu=p.nu_bar;
S=Qxx; L=0; converged=false; Scap=1e12;
for it=1:2000
    Ln=(B'*S*B+Quu)\(B'*S*A);
    Sn=A'*S*A+Qxx-nu*(A'*S*B*Ln);
    if abs(Ln-L)<=1e-12*(1+abs(Ln)); L=Ln; S=Sn; converged=true; return; end
    L=Ln; S=Sn;
    if ~isfinite(S)||S>Scap; converged=false; return; end
end
end

function flag = stability_flag(termEnergy, rmsState)
% Heuristic finite-horizon label from Monte Carlo summaries. A mean-square
% stable loop settles to an O(1-10) energy; an unstable mode inflates the
% terminal energy by many orders of magnitude over the horizon.
if ~isfinite(termEnergy) || termEnergy>1e3 || rmsState>1e2
    flag='unstable';
elseif termEnergy>25 || rmsState>10
    flag='marginal';
else
    flag='stable';
end
end

function fig = plot_case_results(out, caseInfo, outDir)
% Four-panel diagnostic figure: running cost, state and estimate, estimation
% error with the +/- sqrt(P_k) band, and the packet-arrival sequences.
N=out.p.N;
fig=figure('Name',caseInfo.name,'Color','w','Position',[80 60 780 760]);
subplot(4,1,1);
plot(out.k_state, log10(max(out.J_run,realmin)),'LineWidth',1.3);
grid on; xlim([0 N]); xlabel('k'); ylabel('log_{10} J(k)');
title('Running realised cost');
subplot(4,1,2);
stairs(out.k_state,out.x,'LineWidth',1.1); hold on;
stairs(out.k_io,out.xhat,'--','LineWidth',1.1);
grid on; xlim([0 N]); xlabel('k'); ylabel('state');
legend({'x_k','xhat_k'},'Location','best'); title('State and TCP-like estimate');
subplot(4,1,3);

```

```

band=sqrt(max(out.Ppost,0));
plot(out.k_io,out.ee,'LineWidth',1.1); hold on;
plot(out.k_io,band,':','LineWidth',1.0); plot(out.k_io,-band,':','LineWidth',1.0);
grid on; xlim([0 N]); xlabel('k'); ylabel('x_k - xhat_k');
title('Estimation error (dotted: \pm one std. dev. \surd P_k)','Interpreter','tex');
subplot(4,1,4);
yyaxis left; stairs(out.k_io,out.gamma,'LineWidth',1.0); ylim([-0.2 1.2]); ylabel('\gamma_k');
yyaxis right; stairs(out.k_io,out.nu,'LineWidth',1.0); ylim([-0.2 1.2]); ylabel('\nu_k');
xlim([0 N]); xlabel('k'); title('Packet arrivals (\gamma_k sensing, \nu_k actuation)');
if isfield(caseInfo,'title')
    try; sgtitle(caseInfo.title,'FontWeight','bold'); catch; end
end
save_fig(fig, outDir, caseInfo.name);
end

function plot_exercise1_comparison(repOuts, results, Cs, outDir)
% Cross-case overlays: running cost and RMS state by scenario.
nC=numel(repOuts); names=Cs(:,1);
f1=figure('Color','w','Position',[80 80 760 440]); cols=lines(nC); hold on;
for c=1:nC
    plot(repOuts{c}.k_state, log10(max(repOuts{c}.J_run,realmin)), ...
        'LineWidth',1.4,'Color',cols(c,:));
end
grid on; xlabel('k'); ylabel('log_{10} J(k)'); legend(names,'Location','northwest');
title('Exercise 1: running realised cost (representative run per case)');
save_fig(f1,outDir,'ex1_cost_overlay');

f2=figure('Color','w','Position',[80 80 760 440]);
rx=[results.rms_x_mean]; sx=[results.rms_x_std];
bar(rx); hold on; lowErr=min(sx,rx*0.999);
errorbar(1:nC,rx,lowErr,sx,'k','LineStyle','none','CapSize',8);
set(gca,'XTick',1:nC,'XTickLabel',names,'YScale','log');
ylabel('RMS state (MC mean \pm std)'); grid on;
title('Exercise 1: RMS state by case');
save_fig(f2,outDir,'ex1_rms_state_bar');
end

function validate_steady_state_gain(P)
% Compare the steady-state control gain with the forward Riccati-style
% recursion of the provided implementation (nu_bar = 1, A = 1.1). Reports the
% step at which the forward gain matches the steady-state value and the
% residual deviation thereafter.
p=P; p.nu_bar=1;
[Lss,~,~]=tcp_control_gain(p);
A=p.A; B=p.B; Qxx=p.Qxx; Quu=p.Quu; N=p.N;
S=Qxx; Lk=zeros(1,N);
for k=2:N
    Lk(k)=(B'*S*B+Quu)\(B'*S*A);
    S=A'*S*A+Qxx-p.nu_bar*(A'*S*B*Lk(k));
end
dev=abs(Lk(2:N)-Lss);
kConv=find(dev<1e-6,1,'first')+1; % first step within 1e-6 of L_ss
fprintf(['\nController check (A=%.3f, nu_bar=1): steady-state gain L=%.4f; ', ...
    'forward Riccati gain matches it to 1e-6 by step %d; max deviation ', ...
    'after step 5 = %.2e.\n'], A, Lss, kConv, max(dev(5:end)));
end

function save_fig(figHandle, outDir, baseName)
% Save a figure as PNG using a light (print-friendly) theme.
if ~exist(outDir,'dir'); mkdir(outDir); end
try; theme(figHandle,'light'); catch; end
set(figHandle,'Color','w');
ax=findall(figHandle,'Type','axes');
for a=reshape(ax,1,[]); try; a.Toolbar.Visible='off'; catch; end; end
png=fullfile(outDir,[baseName '.png']);
try; exportgraphics(figHandle,png,'Resolution',150);
catch; try; print(figHandle,png,'-dpng','-r150'); catch; saveas(figHandle,png); end; end
end

```

Appendix B: MATLAB code for Exercise 2

The following MATLAB script reproduces the Exercise 2 stability analysis, including the three-dimensional stability landscape. It is self-contained and runs from a clean session, with its supporting routines defined as local functions.

```
%% Appendix B: MATLAB code for Exercise 2
% ELE419 Cybersecurity for Control Systems
% Nijat Badalov (250216213), University of Sheffield, Spring 2026
%
% Mean-square stability boundary of the scalar TCP-like LQG loop under
% denial-of-service packet dropouts, for A = 1.255, N = 100, process-noise
% variance 0.5 and measurement-noise variance 0.15 (ELE419 DoS laboratory
% brief). The stability region in (nu_bar, gamma_bar) is located two ways:
% (i) deterministically, from the boundedness of the control Riccati
% recursion (in nu_bar) and the expected error-covariance recursion
% (in gamma_bar); these give the exact mean-square boundary;
% (ii) by Monte Carlo simulation of the full stochastic loop on a grid and
% on fine one-dimensional sweeps, which supports the trend but is less
% sharp near the threshold. Only base MATLAB is used (no toolboxes).
%
% Run from a clean session: >> Appendix_Code_Exercise2

clear; close all; clc;

masterSeed = 2026;
figDir = fullfile(pwd, 'figures');
if ~exist(figDir, 'dir'); mkdir(figDir); end

% ---- plant and cost (Exercise 2 parameters) -----
P.A=1.255; P.B=1; P.C=1; P.N=100; P.Qxx=1; P.Quu=1;
P.Q=0.5; % process-noise variance Var(w)
P.R=0.15; % measurement-noise variance Var(z)
P.P0=0.01; P.x0=[]; P.validate=false;
crit = 1 - 1/P.A^2; % analytic critical arrival probability

ex2grid = 0:0.05:1; % (nu_bar, gamma_bar) grid for the heatmaps
ex2trials = 80; % Monte Carlo trials per grid point
nb = ex2grid; gb = ex2grid; nN=numel(nb); nG=numel(gb);

% =====
% (i) DETERMINISTIC mean-square stability boundary
% =====
nu_crit_det = bisect_threshold(@(v) ctrl_bounded(setfield(P,'nu_bar',v)), 0,1,1e-3); %#ok<SFLD>
gam_crit_det = bisect_threshold(@(v) cov_bounded( setfield(P,'gamma_bar',v)), 0,1,1e-3); %#ok<SFLD>

ctrlOK=false(1,nN); for i=1:nN; ctrlOK(i)=ctrl_bounded(setfield(P,'nu_bar',nb(i))); end %#ok<SFLD>
covOK =false(1,nG); for j=1:nG; covOK(j) =cov_bounded( setfield(P,'gamma_bar',gb(j))); end %#ok<SFLD>
stableDet = double(covOK(:)*ctrlOK(:).'); % nG x nN, 1 = MS-stable

gf=0:0.01:1; Sdet=nan(size(gf)); Pdet=nan(size(gf));
for i=1:numel(gf)
    [Si,ci]=ctrl_steady(setfield(P,'nu_bar',gf(i))); %#ok<SFLD>
    if ci; Sdet(i)=Si; end
    [Pi,bi]=tcp_expected_covariance(setfield(P,'gamma_bar',gf(i))); %#ok<SFLD>
    if bi; Pdet(i)=Pi; end
end

% =====
% (ii) MONTE CARLO grid: mean state energy, cost, final covariance
% =====
M=ex2trials; logEnergy=nan(nG,nN); meanCost=nan(nG,nN); finalCov=nan(nG,nN);
t0=tic;
for in=1:nN
    p=P; p.nu_bar=nb(in);
    [p.L,~,p.gain_converged]=tcp_control_gain(p); % gain depends on nu_bar only
    for ig=1:nG
        p.gamma_bar=gb(ig);
        sumX2=zeros(1,p.N+1); eJ=zeros(1,M); eP=zeros(1,M);
        for t=1:M
            rng(masterSeed+5000+137*in+t);
            o=simulate_tcp_case(p);
            sumX2=sumX2+o.x.^2; eJ(t)=o.mean_cost; eP(t)=o.P_final;
        end
        V=sumX2/M;
        logEnergy(ig,in)=log10(max(mean(V),realmin));
        meanCost(ig,in)=mean(eJ); finalCov(ig,in)=mean(eP);
    end
end
fprintf('MC grid: %d x %d points, %d trials each (%.1f s).\n', nN,nG,M,toc(t0));
```

```

% fine 1-D sweeps at two horizons
Ns=[P.N, 300];
sweep_nu = run_sweep(P, gf, 'nu', 1.0, Ns, masterSeed);
sweep_gam = run_sweep(P, gf, 'gamma', 1.0, Ns, masterSeed);

% =====
% FIGURES
% =====
f1=figure('Color','w','Position',[60 60 1000 430]);
subplot(1,2,1); draw_heatmap(nb,gb,stableDet,crit,'',false);
title('(A) Deterministic MS-stable region');
subplot(1,2,2); draw_heatmap(nb,gb,logEnergy,crit,'log_{10} mean state energy',true);
title('(B) Monte Carlo log_{10} E[(1/N)\Sigma x_k^2]');
save_fig(f1,figDir,'ex2_stability_heatmaps');

f2=figure('Color','w','Position',[60 60 1000 430]);
subplot(1,2,1); draw_heatmap(nb,gb,log10(max(meanCost,realmin)),crit,'log_{10} mean cost',true);
title('Cost map: log_{10} mean per-step cost');
subplot(1,2,2); draw_heatmap(nb,gb,log10(max(finalCov,realmin)),crit,'log_{10} final covariance',true);
title('Estimator: log_{10} final P (varies with \gamma only)');
save_fig(f2,figDir,'ex2_cost_covariance_heatmaps');

f3=figure('Color','w','Position',[60 60 1000 430]);
subplot(1,2,1); plot_sweep(sweep_nu,crit,'\nu_{bar} (\gamma_{bar}=1)'); title('Vary actuation arrival (MC)');
subplot(1,2,2); plot_sweep(sweep_gam,crit,'\gamma_{bar} (\nu_{bar}=1)'); title('Vary sensing arrival (MC)');
save_fig(f3,figDir,'ex2_threshold_sweeps');

f4=figure('Color','w','Position',[60 60 1000 430]);
subplot(1,2,1); semilogy(gf,Sdet,'LineWidth',1.6); hold on; yl=ylim;
plot([crit crit],yl,'k--','LineWidth',1.2); grid on;
xlabel('\nu_{bar}'); ylabel('steady-state control value S');
legend({'S(\nu_{bar})',sprintf('1-1/A^2=%3f',crit)},'Location','northeast');
title('Control recursion diverges as \nu_{bar}\downarrow 1-1/A^2');
subplot(1,2,2); semilogy(gf,Pdet,'LineWidth',1.6); hold on; yl=ylim;
plot([crit crit],yl,'k--','LineWidth',1.2); grid on;
xlabel('\gamma_{bar}'); ylabel('steady-state expected covariance P');
legend({'P(\gamma_{bar})',sprintf('1-1/A^2=%3f',crit)},'Location','northeast');
title('Expected covariance diverges as \gamma_{bar}\downarrow 1-1/A^2');
save_fig(f4,figDir,'ex2_deterministic_recursions');

% supporting 3D stability landscape: log10 mean state energy as a surface, with
% a base contour (surfc) and thin dashed threshold lines on the base plane.
% The surface is lightly smoothed and interpolated for DISPLAY ONLY; this does
% not affect any reported threshold, which comes from the recursions above.
[NBb,GBb]=meshgrid(nb,gb);
Zs=smooth2(logEnergy); % light 3x3 mean
nbf=linspace(0,1,101); gbf=linspace(0,1,101); [NBf,GBf]=meshgrid(nbf,gbf);
Zf=interp2(NBb,GBb,Zs,NBf,GBf,'spline'); % smooth display surface
f5=figure('Color','w','Position',[80 80 780 560]);
surfc(nbf,gbf,Zf,'EdgeColor','none'); shading interp; hold on;
colormap(gca,seqmap); clim([pct(Zf,3) pct(Zf,97)]); % percentile colour clip
cb=colorbar; cb.Label.String='log_{10} mean state energy';
zb=min(Zf(:))-0.5; zlim([zb max(Zf(:))]);
plot3([crit crit],[0 1],[zb zb],'k--','LineWidth',1.4); % nu_bar = 0.365 (base)
plot3([0 1],[crit crit],[zb zb],'k--','LineWidth',1.4); % gamma_bar = 0.365 (base)
xlabel('\nu_{bar} (actuation arrival prob.)');
ylabel('\gamma_{bar} (sensing arrival prob.)');
zlabel('log_{10} mean state energy');
title(sprintf('3D stability landscape (dashed thresholds at \nu_{bar},\gamma_{bar}\approx%3f',crit));
view(-38,26); grid on; box on; pbaspect([1 1 0.6]);
save_fig(f5,figDir,'ex2_stability_3d');

% =====
% SUMMARY (console + CSV)
% =====
fprintf('\nAnalytic critical arrival probability 1-1/A^2 = %4f\n', crit);
fprintf('Deterministic nu threshold (control recursion) = %4f\n', nu_crit_det);
fprintf('Deterministic gamma threshold (covariance recursion) = %4f\n', gam_crit_det);
fprintf('Monte Carlo nu threshold : N=%d ~ %3f , N=%d ~ %3f\n', ...
    Ns(1),sweep_nu.thr_short,Ns(2),sweep_nu.thr_long);
fprintf('Monte Carlo gamma threshold : N=%d ~ %3f , N=%d ~ %3f\n', ...
    Ns(1),sweep_gam.thr_short,Ns(2),sweep_gam.thr_long);
fprintf('=> mean-square stable when nu_bar > %3f AND gamma_bar > %3f.\n', crit, crit);

q={'analytic_1_minus_1overA2';'det_nu_threshold';'det_gamma_threshold'; ...
    'mc_nu_threshold_N100';'mc_nu_threshold_N300';'mc_gamma_threshold_N100';'mc_gamma_threshold_N300'};
v=[crit; nu_crit_det; gam_crit_det; sweep_nu.thr_short; sweep_nu.thr_long; ...
    sweep_gam.thr_short; sweep_gam.thr_long];
writetable(table(q,v,'VariableNames',{'quantity','value'}), ...
    fullfile(figDir,'exercise2_thresholds.csv'));

fprintf('\nExercise 2 complete. Figures and exercise2_thresholds.csv saved in: %s\n', figDir);

```

```

%% ===== local functions =====

function out = simulate_tcp_case(p)
% One realisation of the scalar TCP-like LQG loop (see Appendix A header).
A=p.A; B=p.B; C=p.C; N=p.N; Qxx=p.Qxx; Quu=p.Quu;
if isfield(p,'L') && ~isempty(p.L)
    L=p.L;
    if isfield(p,'gain_converged'); gc=p.gain_converged; else; gc=NaN; end
else
    [L,~,gc]=tcp_control_gain(p);
end
nu=double(rand(1,N)<p.nu_bar); gamma=double(rand(1,N)<p.gamma_bar);
w=sqrt(p.Q)*randn(1,N); z=sqrt(p.R)*randn(1,N);
if isempty(p.x0); x0=sqrt(p.P0)*randn; else; x0=p.x0; end
x=zeros(1,N+1); x(1)=x0; xhat=zeros(1,N+1); u=zeros(1,N); y=zeros(1,N);
Ppost=zeros(1,N+1); Ppost(1)=p.P0; mu0=0;
y(1)=gamma(1)*C*x(1)+z(1); s0=C*p.P0*C'+p.R;
if gamma(1)==1 && s0>0
    K0=(p.P0*C')/s0; xhat(1)=mu0+K0*(y(1)-C*mu0); Ppost(1)=p.P0-K0*C*p.P0;
else
    xhat(1)=mu0; Ppost(1)=p.P0;
end
u(1)=-L*xhat(1); x(2)=A*x(1)+nu(1)*B*u(1)+w(1);
for k=1:N-1
    j=k+1; y(j)=gamma(j)*C*x(j)+z(j);
    [xhat(j),Ppost(j)]=kalman_tcp_update(p,y(j),xhat(k),u(k),nu(k),gamma(j),Ppost(k));
    u(j)=-L*xhat(j); x(j+1)=A*x(j)+nu(j)*B*u(j)+w(j);
end
ee=x(1:N)-xhat(1:N);
stage=Qxx*x(1:N).^2+nu.*(Quu*u.^2);
J_realised=Qxx*x(N+1)^2+sum(stage);
out.x=x; out.ee=ee; out.nu=nu; out.gamma=gamma; out.gain_converged=gc;
out.J_realised=J_realised; out.mean_cost=J_realised/N;
out.emp_nu=mean(nu); out.emp_gamma=mean(gamma);
out.rms_x=sqrt(mean(x.^2)); out.rms_e=sqrt(mean(ee.^2));
out.max_absx=max(abs(x)); out.xN2=x(N+1)^2; out.P_final=Ppost(N);
end

function [xhat_upd,P_upd] = kalman_tcp_update(p,y,xhat_prev,u_prev,nu_prev,gamma_cur,P_prev)
% TCP-like Kalman update: prediction uses the known previous actuation outcome
% nu_prev; the current sensing arrival gamma_cur gates the correction.
A=p.A; B=p.B; C=p.C; Q=p.Q; R=p.R;
xhat_pred=A*xhat_prev+nu_prev*B*u_prev; P_pred=A*P_prev*A'+Q;
innov=C*P_pred*C'+R;
if gamma_cur==1 && all(innov(:)>0)
    K=(P_pred*C')/innov; xhat_upd=xhat_pred+K*(y-C*xhat_pred);
    P_upd=P_pred-K*C*P_pred; P_upd=0.5*(P_upd+P_pred);
else
    xhat_upd=xhat_pred; P_upd=P_pred;
end
end

function [L,S,converged] = tcp_control_gain(p)
% Steady-state TCP-like LQG gain from the control Riccati recursion
%  $L=(B'SB+Quu)^{-1}B'SA$ ,  $S=A'SA+Qxx-nu\_bar A'SB L$ .
A=p.A; B=p.B; Qxx=p.Qxx; Quu=p.Quu; nu=p.nu_bar;
S=Qxx; L=0; converged=false; Scap=1e12;
for it=1:2000
    Ln=(B'*S*B+Quu)\(B'*S*A); Sn=A'*S*A+Qxx-nu*(A'*S*B*Ln);
    if abs(Ln-L)<=1e-12*(1+abs(Ln)); L=Ln; S=Sn; converged=true; return; end
    L=Ln; S=Sn;
    if ~isfinite(S)||S>Scap; converged=false; return; end
end
end

function [Sss,bounded] = ctrl_steady(p)
% Steady-state control value S, iterated long enough to resolve the threshold.
% bounded is false when nu_bar is below the critical arrival probability.
A=p.A; B=p.B; Qxx=p.Qxx; Quu=p.Quu; nu=p.nu_bar; S=Qxx; cap=1e8; bounded=true;
for it=1:1e5
    Ln=(B'*S*B+Quu)\(B'*S*A); Sn=A'*S*A+Qxx-nu*(A'*S*B*Ln);
    if abs(Sn-S)<=1e-12*(1+abs(Sn)); Sss=Sn; bounded=true; return; end
    S=Sn;
    if ~isfinite(S)||S>cap; Sss=S; bounded=false; return; end
end
Sss=S; bounded=isfinite(S)&&S<=cap;
end

function tf = ctrl_bounded(p)
% True iff a stabilising steady-state controller exists (control recursion bounded).
[~,tf]=ctrl_steady(p);
end

```



```

function [Pbar,bounded] = tcp_expected_covariance(p,maxiter,cap)
% Steady-state expected error-covariance recursion (intermittent observations):
% Pbar <- A Pbar A' + Q - gamma_bar (A Pbar C')(C Pbar C'+R)^{-1}(C Pbar A').
% Finite fixed point iff gamma_bar > 1 - 1/A^2.
if nargin<2||isempty(maxiter); maxiter=1e5; end
if nargin<3||isempty(cap); cap=1e8; end
A=p.A; C=p.C; Q=p.Q; R=p.R; g=p.gamma_bar; Pp=Q; bounded=true;
for it=1:maxiter
    innov=C*Pp*C'+R; corr=g*(A*Pp*C')*(innov\((C*Pp*A')));
    Pn=A*Pp*A'+Q-corr;
    if abs(Pn-Pp)<=1e-12*(1+abs(Pn)); Pbar=Pn; bounded=true; return; end
    Pp=Pn;
    if ~isfinite(Pp)||Pp>cap; Pbar=Pp; bounded=false; return; end
end
Pbar=Pp; bounded=isfinite(Pp)&&Pp<=cap;
end

function tf = cov_bounded(p)
% True iff the expected error-covariance recursion stays bounded.
[~,tf]=tcp_expected_covariance(p,1e5,1e8);
end

function thr = bisect_threshold(testFun, lo, hi, tol)
% Smallest value in [lo,hi] for which the monotone testFun is true (bisection).
if ~testFun(hi); thr=NaN; return; end
if testFun(lo); thr=lo; return; end
while hi-lo>tol; mid=0.5*(lo+hi); if testFun(mid); hi=mid; else; lo=mid; end; end
thr=hi;
end

function s = run_sweep(P, gf, which, fixedVal, Ns, masterSeed)
% One-dimensional Monte Carlo sweep of one arrival probability at two horizons,
% returning the log mean state energy and a finite-horizon growth threshold.
M=150; logE=nan(numel(Ns),numel(gf)); grow=nan(numel(Ns),numel(gf));
for hi=1:numel(Ns)
    p=P; p.N=Ns(hi);
    for i=1:numel(gf)
        if strcmp(which,'nu'); p.nu_bar=gf(i); p.gamma_bar=fixedVal;
        else; p.gamma_bar=gf(i); p.nu_bar=fixedVal; end
        [p.L,~,p.gain_converged]=tcp_control_gain(p);
        sumX2=zeros(1,p.N+1);
        for t=1:M
            rng(masterSeed+9000+311*hi+17*i+t);
            o=simulate_tcp_case(p); sumX2=sumX2+o.x.^2;
        end
        V=sumX2/M; logE(hi,i)=log10(max(mean(V),realmin));
        midI=round(p.N/2)+1; grow(hi,i)=V(end)/max(V(midI),realmin);
    end
end
s.gf=gf; s.Ns=Ns; s.logE=logE; s.grow=grow;
s.thr_short=first_stable(gf,grow(1,:)<3); s.thr_long=first_stable(gf,grow(2,:)<3);
s.which=which;
end

function thr = first_stable(grid, stableRow)
% Smallest grid value at which the row becomes (and stays) stable.
thr=NaN;
for i=1:numel(grid)
    if stableRow(i) && all(stableRow(i:end)); thr=grid(i); return; end
end
end

function draw_heatmap(nb, gb, Z, crit, cbarLabel, useColorbar)
% Heatmap over (nu_bar, gamma_bar) with the analytic critical lines 1 - 1/A^2.
imagesc(nb,gb,Z); set(gca,'YDir','normal'); hold on;
xlabel('\nu_{bar} (actuation arrival)'); ylabel('\gamma_{bar} (sensing arrival)');
if useColorbar; cb=colorbar; cb.Label.String=cbarLabel; end
plot([crit crit],[gb(1) gb(end)],'w--','LineWidth',1.5);
plot([nb(1) nb(end)],[crit crit],'w--','LineWidth',1.5);
axis([nb(1) nb(end) gb(1) gb(end)]);
end

function plot_sweep(s, crit, xlab)
% log mean state energy vs arrival probability for both horizons.
plot(s.gf,s.logE(1,:),'LineWidth',1.5); hold on; plot(s.gf,s.logE(2,:),'LineWidth',1.5);
yl=ylim; plot([crit crit],yl,'k--','LineWidth',1.2); grid on;
xlabel(xlab); ylabel('log_{10} mean state energy');
legend({sprintf('N=%d',s.Ns(1)),sprintf('N=%d',s.Ns(2)),sprintf('1-1/A^2=%.3f',crit)}, ...
    'Location','northeast');
end

function save_fig(figHandle, outDir, baseName)

```

```

% Save a figure as PNG using a light (print-friendly) theme.
if ~exist(outDir,'dir'); mkdir(outDir); end
try; theme(figHandle,'light'); catch; end
set(figHandle,'Color','w');
ax=findall(figHandle,'Type','axes');
for a=reshape(ax,1,[]); try; a.Toolbar.Visible='off'; catch; end; end
png=fullfile(outDir,[baseName '.png']);
try; exportgraphics(figHandle,png,'Resolution',150);
catch; try; print(figHandle,png,'-dpng','-r150'); catch; saveas(figHandle,png); end; end
end

function C = seqmap()
% Sequential light-blue to deep-navy colour map (perceptually increasing and
% still legible when printed in grey scale). Used for the energy landscape.
a=[0.984 0.992 1.000; 0.74 0.85 0.93; 0.33 0.58 0.79; 0.13 0.34 0.55; 0.03 0.16 0.33];
C=interp1(linspace(0,1,size(a,1)), a, linspace(0,1,256));
end

function p = pct(x,q)
% Percentile of x at q in [0,100], computed by sorting (no toolbox needed).
x=sort(x(:)); n=numel(x); p=x(max(1,min(n,round(q/100*n))));
end

function Zs = smooth2(Z)
% Edge-correct 3x3 moving average for display smoothing of the energy grid.
K=ones(3); Zs=conv2(Z,K,'same')./conv2(ones(size(Z)),K,'same');
end

```